CM1010 Introduction to Programming II - Log Sheets for Final Project

NGAI ChiHung, Student Number 220227964

March 9, 2024

Topic 7 Log Sheet

Project Title

Music Visualisation Project

Topic

Topic 7 - Extending the Apps

What progress have you made this topic?

1. Update the MP3 player

- volume control has been added onto the mp3 player
  - instead of using slider, which needs CSS code for styling, I have created two rectangles to represent the sound volume
- change of key configs
  - ”m” for popping out or hiding the visual selection menu
  - space bar’s new use, for playing and pausing music
  - ”f” key for playing faster, ”s” key for playing slower
  - use switch...case block to tidy up the keyPressed button selections
- specified some of the visuals’ music to play
  - when the visual is started, its relevant music will start at the same time

2. Update the backgrounds

- the same background coloring mechanism as the first visual
- visual 2 (The Sun) - add circular background with fixed color pattern
- visual 3 (The Rainbow) - add diagonal background with color varied with the fourier.getEnergy values
- visual 4 (The Garden) - add diagonal background with some variations on the green part

3. Add one more visual

- One more visual (The Disco) has been added, with adding background

4. Tidy up the code

- sketch.js has contain codes for initiating the star array (for the first visual)
- to meke the code more easy to follow, I have move those into wavepattern.js object

What problems have you faced and were you able to solve them?

1. Volume control

- I want to have a new volume control on the player
- slider can do this, but styling needs CSS code
- I want to avoid CSS code in the canvase (i.e. try using p5.js code as much as I can)
- therefore, I made two rectangles to relace the slider, with adding arrow up and arrow down buttons to control the volume

Are you on target to successfully complete your project? If you aren't on target, how will you address the issue?

- My project is a little bit behind the schedule
- It is because I looked for the web for studying methods of constructing the MP3 player and the music visualisation in more time effective and beautiful way
- starting on next week, I will be focusing the visualisations 1 to 4, for meeting the project progress

What are you planning to do over the next few weeks?

1. Visual 1 - North Pole Light
    - remove the Mountain visuals
    - replace by the mountains to the terrain the reflect sound property
2. Visual 2 - The Sun
    - update of this visual has almost been finished
3. Visual 3 - The Rainbow
    - the background update has finished

- need to update the progress circular bars from five colors into seven colors

4. Visual 4 - Garden
    - the background has become green
    - i have studied more on my reference book, one more object (tree) can be added next week, to make the visual more motion
5. Visual 5 - Disco
    - this is an additional visual
    - I am aiming to add a cup of cocktail, then in the middle of song some light beans shine
    - initial stage (building the functional constructor for this visual, and adding background) has finished
    - in next steps I will add the cup, cocktail with bubble, and disco lights
6. Visual 6 - Performance Platform
    - Review of this visual for project additional visual is in progress
    - it includes the workload, time allowed for this part

Except from buinding up the visuals and the mp3 player, I will also arrange the following

- add Async and Await onto visuals
- arrange tests to the project
- prepare the report

Appendix - The summary of code I have added

Additional MP3 player codes

Below is the capture from the MP3 player sound control slide

```
/* new button for volume control */
this.buttons = {
  /* codes for other buttons skipped */
  // sound control slider
  volumeControlSlider: new Button(
    divPanel.size().width / 2 + 300,
    height - 80,
    150,
    20,
    new VolumeControlDrawStrategy(),
    new VolumeControlCheckStrategy()
  ),
};
```

```
14
15 /* in controlsAndInput.js */
16
17 // responds to keyboard presses, keycode the ascii code of the keypressed
18 keyPressed = function (keycode) {
19   // when number keys are pressed, select the visuals
20
21   switch (keycode) {
22     /* codes for other buttons skipped */
23
24     // up button - make the volume louder
25     case 38:
26       playVolume += 0.1;
27       if (playVolume > 1.0) playVolume = 1.0;
28       controls.buttons.volumeControlSlider.drawStrategy.setSliderPos(
29         playVolume
30       );
31       sound.setVolume(playVolume);
32       break;
33
34     // down button - make the volume quieter
35     case 40:
36       playVolume -= 0.1;
37       if (playVolume <= 0.1) playVolume = 0.1;
38       controls.buttons.volumeControlSlider.drawStrategy.setSliderPos(
39         playVolume
40       );
41       sound.setVolume(playVolume);
42       break;
43   }
44 };
45
46 /* in volumeControlButton.js */
47
48 // code for drawing out the volume control slider
49 class VolumeControlDrawStrategy {
50   constructor() {
51     this.x = null;
52     this.y = null;
53     this.width = null;
54     this.height = null;
55     this.sliderPos = 0.1;
56   }
57
58   draw() {
59     this.y = height - 80;
60     push();
61     stroke(255, 246, 224);
```

```
    fill(216, 217, 218);
    rect(this.x, this.y, this.width, this.height);
    stroke(97, 103, 122);
    fill(136, 196, 255);
    rect(this.x + this.width * this.sliderPos - 10, this.y - 5, 20, 30);
    pop();
  }

  // update this position when volume changes
  setSliderPos(soundLevel) {
    this.sliderPos = soundLevel;
  }
}
```

Sample code of visual backgrounds

Below is capture from the wavepattern.js

```
// make the background looks like near the sun
// RadialGradient is not p5.js function, this is WebAPI function
// https://developer.mozilla.org/en-US/docs/Web/API/CanvasRenderingContext2D/createRadialGradient

let drawWarmBackground = function () {
  let gradient = drawingContext.createRadialGradient(
    width / 2,
    height / 2,
    60,
    width / 2,
    height / 2,
    height / 2
  );

  gradient.addColorStop(0, color(238, 93, 95));
  gradient.addColorStop(0.05, color(231, 126, 110));
  gradient.addColorStop(0.1, color(231, 62, 101));
  gradient.addColorStop(0.15, color(146, 0, 4));
  gradient.addColorStop(1, color(90, 6, 0));
  drawingContext.fillStyle = gradient;

  rect(0, 50, width, height - 50 - 120);
};
```

Topic 8 Log Sheet

Project Title

Music Visualisation Project

Topic

Topic 8 - Async and Callbacks

What progress have you made this topic?

1. Update the visuals

- Visual 3 - Update the progress bars from five colors to seven colors
- Visual 4 - Add two trees which branched varied by the music
- Visual 5 - Add the cocktail (with bubbles) and lyrics for the visual

2. Preparing the test cases

- set up integration test and unit test cases

3. Prepare the report

- study the questions and items to be replied for the report

What problems have you faced and were you able to solve them?

1. Visualising the music frequency map

- in The Rainbow visual, the fourier.getEnergy function has five string arguments, as per p5.sound documentation
- but the rainbow has seven colors
- to make the sound frequency spectrum divide by seven spectrum, another argument input method has to be complied

- the getEnergy(min, max) input method will generates out energy of sound between min frequency and max frequency
- by dividing human-recognisable sound frequencies into seven ranges (ignore 16kHz - 32kHz which is too high for my songs), sound energy can be extracted and being mapped onto the status bar of each of the seven colors
- hence the rainbow can be generated

2. Adding the trees

- in Garden visual, since the original phyllotaxis part is not filling in the screen
- I have added two trees which is out of the mid-term project plan
- here, and old programming language Logo is being remake (in the Turtle class)
  - a turtle can leave a track (this is drawing) when it is walking on the screen
  - the turtle can move forward, backward and turn for each step
  - more detail can make reference to `https://en.wikipedia.org/wiki/Logo_(programming_language)`
- data structure stack is used to store its steps, for moving backward for drawing branches in other way
- hence, this tree, with two branches on each node, can be donw by making the turtle to and fro, with the use of stack pop and stack push functions
- below shows recursion codes of the movement of turtle movements

3. Adding lyrics

- For making the Disco visual more attaractive, I have added the lyrics show, on the top of the cocktail cup
- this is actually calling p5.js loadTable function (this is for data visualisation app), to read in the lyrics and their display time (in ms)
- As the song is Korean song, one more font is being used to display the lyrics

Are you on target to successfully complete your project? If you aren't on target, how will you address the issue?

- My project is a little bit behind the schedule
- the terrain in the first visual will be finished in next log report
- the test cases and report will start from next log
- Async and Await is being delayed due to the making of other visual parts

What are you planning to do over the next few weeks?

1. Visual 1 - North Pole Light
    - remove the Mountain visuals
    - replace by the mountains to the terrain the reflect sound property
2. Visual 2 - The Sun
    - update of this visual has almost been finished
3. Visual 3 - The Rainbow
    - update of this visual has almost been finished
4. Visual 4 - Garden
    - update of this visual has almost been finished
5. Visual 5 - Disco
    - cocktail cup, bubble and lyrics has finished
    - in next steps I will add the disco lights
6. Visual 6 - Performance Platform
    - after review the project scope and project timeline, this has to be withdrawn from my production plan

Except from buinding up the visuals and the mp3 player, I will also arrange the following

- add Async and Await onto visuals
- arrange tests to the project
- prepare the report

Appendix - The summary of code I have added

Make the Rainbow visual more complete

Below is the capture of The Rainbow that capture sound energy within assigned frequencies spectrum, into energy value for rainbow visual.

```
/* in needles.js */

function Needles() {
  /* code skipped */
  // frquencies used by the energyfunction to retrieve a value for each plot
```

```
  // human detectable sound frequencies are ranged between 16Hz and 32,768 Hz
  // 16,384 Hz - 32,768 Hz are too high that not included in the songs
  // hence i use the other ranges to divide into seven ranges for illustrating rainbow visual
  this.frequencyBins = [
    { lower: 16, upper: 32 },
    { lower: 32, upper: 280 },
    { lower: 280, upper: 512 },
    { lower: 512, upper: 2048 },
    { lower: 2048, upper: 8192 },
    { lower: 8192, upper: 12288 },
    { lower: 12288, upper: 16384 },
  ];
  this.rainbow = [
    color(255, 26, 26),
    color(265, 165, 0),
    color(255, 255, 26),
    color(0, 200, 0),
    color(0, 194, 222),
    color(0, 65, 141),
    color(95, 40, 211),
  ];

  // draw the plots to the screen
  this.draw = function () {
    /* code skipped */
    // create an array amplitude values from the fft.
    let spectrum = fourier.analyze();

    for (let i = 0; i < this.rainbow.length; i++) {
      // getEnergy with two digit variables will get averaged amplitude of range of sound frequencies
      energy = fourier.getEnergy(
        this.frequencyBins[i].lower,
        this.frequencyBins[i].upper
      );
    }
    /* code skipped */
  };
}
```

Dawing of tree in Garden visual

Below is the capture of how the tree is being drawn by Logo Turtle.

```
/* in phyllotaxis.js */

// simulation of Logo language turtle
class Turtle {
```

```
  constructor(x = 0, y = 0, heading = 0) {
    this.coordinateVector = createVector(x, y);
    this.headingVector = createVector(1, 0).rotate(heading);
    this.stateStack = [];
  }

  // return the location of the turtle
  coordinate() {
    return this.coordinateVector.copy();
  }

  // ask the turgle more forward (can backword)
  // return a vector with from and destination coordinates
  forward(length) {
    const from = this.coordinate();
    const v = p5.Vector.mult(this.headingVector, length);

    this.coordinateVector.add(v);
    const to = this.coordinate();

    return { from, to };
  }

  // ask the turtle to turn
  turn(angle) {
    this.headingVector.rotate(angle);
  }

  // store steps into memory stack
  pushStep() {
    this.stateStack.push({
      x: this.coordinateVector.x,
      y: this.coordinateVector.y,
      heading: this.headingVector.heading(),
    });
  }

  // remove from the memory stack the steps
  popStep() {
    const { x, y, heading } = this.stateStack.pop();

    this.coordinateVector.x = x;
    this.coordinateVector.y = y;
    this.headingVector.setHeading(radians(heading));
  }
}

// drawing the tree with the use of turtle
```

```
function Phyllotaxis() {
  this,
    (draw = function () {
      /* codes skipped */
      // draw the tree on left
      push();
      translate(width / 4, height - 120);
      rotate(-90);
      this.drawTree(100);
      pop();

      // draw the tree on right
      push();
      translate((width * 5) / 6, height - 120);
      rotate(-90);
      this.drawTree(120);
      pop();
    });

  // draw the tree
  // this is recursion, until n = 0
  this.drawTree = function (
    length,
    angle = 37,
    branchRatio = 0.75,
    t = new Turtle(),
    n = 10
  ) {
    // tree body needs to be brown
    if (n == 10 || n == 9) {
      strokeWeight(10);
      stroke(color(140, 103, 62));
    }
    // leaves needs to be green
    else {
      strokeWeight(2);
      stroke(color(9, 255, 0));
    }

    // ask the turtle to move forward (for drawing)
    footprint(t.forward(length));

    if (n > 0) {
      t.pushStep();

      t.turn(angle);
      this.drawTree(
        length *
```

```
          branchRatio *
          map(fourier.getEnergy("lowMid"), 0, 255, 0.85, 1.15),
        angle * random(0.95, 1.05),
        branchRatio,
        t,
        n - 1
      );

      t.popStep();

      t.turn(-angle);
      this.drawTree(
        length *
          branchRatio *
          map(fourier.getEnergy("highMid"), 0, 255, 0.85, 1.15),
        -angle * random(0.95, 1.05),
        branchRatio,
        t,
        n - 1
      );
    }
  };
}
```

Showing the Lyrics

Below is the code for loading csv file to get lyrics and then display them in the Disco visual

```
/* in disco.js */

// light plate for showing lyrics
class LightPlate {
  #lyrics = [];
  #loadedTable;
  /* code skipped */

  constructor(x, y) {
    this.#loadedTable = loadTable("../assets/lyric.csv", "csv", () =>
      this.onFileload()
    );
  }

  // load in lyrics from csv file
  onFileload() {
    for (let row = 1; row < this.#loadedTable.getRowCount(); row++)
      for (
        let column = 1;
```

```
20        column < this.#loadedTable.getColumnCount();
21        column++
22      )
23        this.#lyrics.push({
24          time: this.#loadedTable.getString(row, 0),
25          lyric: this.#loadedTable.getString(row, 1),
26        });
27    }
28  }
```